S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

To create competent and creative professional in the field of Data Science to address the data-driven decision-making for the benefits of industry and society

Personal Expense Tracker

Personal Expense Tracker: Develop a Python-based application that helps users record, organize and monitor their daily income and expenses. The system allows users to add, view, update and delete transactions, categorize expenses, calculate total income, total expenses and current balance, and generate monthly or category-wise reports, with records stored securely using file handling.

NAME
OM MOHAN MOUNDEKAR

ROLL NO.
CD25026

YEAR / SEMESTER
2nd / 3rd SEM

GUIDED BY
Ashwini Yerlekar Ma'am

SUBJECT
Python Programming Lab (N-PCCCD304P)

WHAT THIS PROJECT COVERS

Project Modules Overview

01

MODULE 1

Transaction Data & Record Management

Add, view, update and delete transactions; categorize them and store data using file handling.



02

MODULE 2

Code Development and Calculations

Validate input, calculate total income, total expenses and balance; flowcharts and working of each step.



03

MODULE 3

Report Generation and Testing

Generate monthly and category-wise expense reports; validate and test every scenario.



Module 1: Transaction Data & Record Management



Purpose of Module 1

Build a system that can store and manage every income and expense entry.



Why records matter

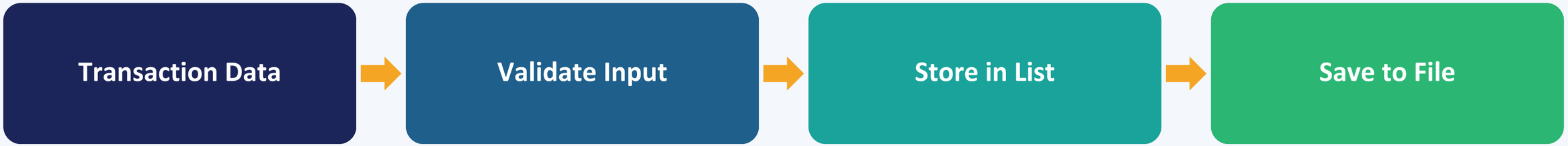
Correct, organised records are the base for calculating totals and reports later.



How Python helps

Python stores, updates and retrieves financial records quickly and accurately.

MODULE 1 FLOW



WHAT WE STORE

Transaction Information

Every transaction record stores these basic details:

- **Transaction ID**
- **Category**
- **Date**
- **Amount**
- **Type (Income / Expense)**
- **Note**

Categories

Expense: Food, Travel, Rent, Bills, Shopping, Health, Education, Entertainment, Other.
Income: Salary, Freelance, Pocket Money, Gifts, Investments, Other.

EXAMPLE RECORD

Transaction ID	**101**
Date	**2026-09-12**
Type	**Expense**
Category	**Food**
Amount	**450.00**
Note	**Lunch with friends**

THE FIVE BASIC ACTIONS

Record Management Operations

ADD



Add a new income or expense entry

VIEW



Display all, monthly or category records

UPDATE



Modify an existing transaction by ID

DELETE



Remove a transaction after confirmation

CATEGORIZE



Tag each entry with a category

TYPICAL FLOW

Add → **View** → **Update / Delete** → **Save to File**

UNDER THE HOOD

Python Concepts Used

Variables

Hold values like amount, category and date.

Lists / Dictionaries

A list stores all records; each one is a dictionary like {"id": 101, "amount": 450}.

Functions

Reusable blocks like add_transaction() or show_summary().

Conditional Statements

if amount > 0: → accept the entry, else ask again.

Loops

A while loop shows the menu; for loops scan records.

File Handling

Opens a file to save or read transaction data.

JSON /CSV Module

json.dump() and json.load() keep records permanently.

Exception Handling

try / except stops crashes on wrong input.

Data Storage & Module 1 Summary



- Records remain available even after closing the program
- Data can be viewed and updated again later
- File handling reduces manual record keeping

Module 1 creates the foundation for code development and calculations in Module 2

MODULE 2 OVERVIEW

What Does Module 2 Cover?

CRUD Operations

- Create / Add transactions
- Read / View / Filter records
- Update by transaction ID
- Delete with confirmation

Business Logic

- Validate amount, date, type
- Calculate total income
- Calculate total expenses
- Calculate current balance

Result Processing

- Group records by month
- Group expenses by category
- Find % share of each category
- Show final summary

MODULE 2 FLOW



CRUD Operations in Expense Management

Operation	Meaning	Project Function
Create	Add new transaction	`add_transaction()`
Read	View / filter records	`view_transactions()`
Update	Modify existing data	`update_transaction()`
Delete	Remove a record	`delete_transaction()`

CRUD lets the user manage transactions efficiently — every menu option maps to one of these four actions.

RECORD LIFECYCLE



Business Logic of the System

> *“Business logic is the set of rules used by the program to process financial data and produce correct totals.”*

- Accept valid transaction details
- Accept amount, date and category
- Calculate total income
- Calculate total expenses

- Calculate current balance
- Warn when balance is negative
- Reject invalid input and ask again
- **If amount ≤ 0 or date is invalid, reject the entry**

PROCESSING FLOW

Transaction Data → **Processing Rules** → **Final Summary**

MODULE 2

Calculation Logic

TOTAL INCOME

= Sum of all income amounts

TOTAL EXPENSES

= Sum of all expense amounts

CURRENT BALANCE

= Total Income − Total Expenses

WORKED EXAMPLE

Income – Salary	**35,000**
Expense – Rent	**9,000**
Expense – Food	**4,500**
Expense – Travel	**2,200**
Total Expenses	**15,700**

35,000 − 15,700

= ₹ 19,300

Current Balance

CALCULATION FLOW

Records → **Total Income** → **Total Expenses** → **Balance** → **Display**

Validation Rules & Input Checks

Rule	Condition	Action
Amount	Number greater than 0	Ask again
Type	1 = Income, 2 = Expense	Ask again
Category	Number from the shown list	Ask again
Date	YYYY-MM-DD (Enter = today)	Ask again
Transaction ID	Must exist in the file	“ID not found.”
Delete	Confirm with y / n	Cancel if not y
Balance < 0	Spending exceeds income	Warning shown

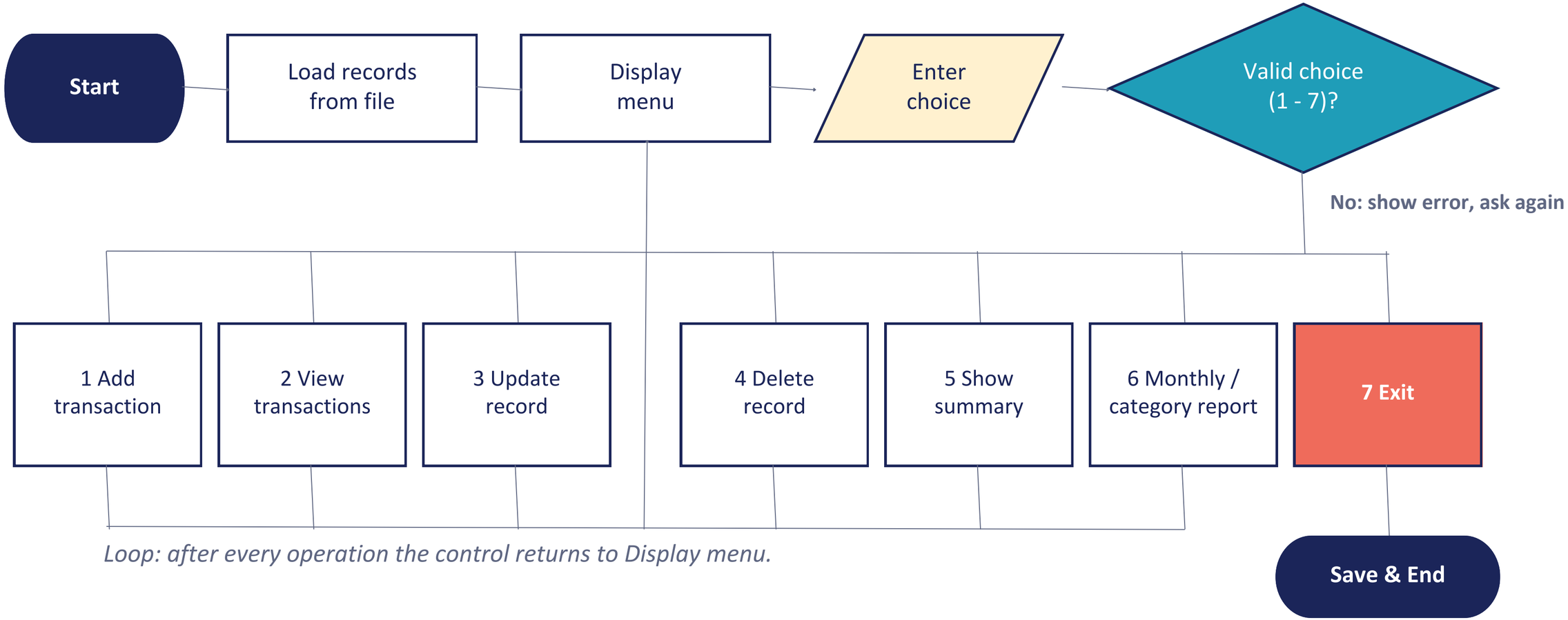
ENTRY RULE

ACCEPT → every field valid
REJECT → any field invalid

get_amount()

```
while True:
    try:
        amount = float(input())
        if amount > 0:
            return amount
        print("Must be > 0")
    except ValueError:
        print("Enter a number")
```

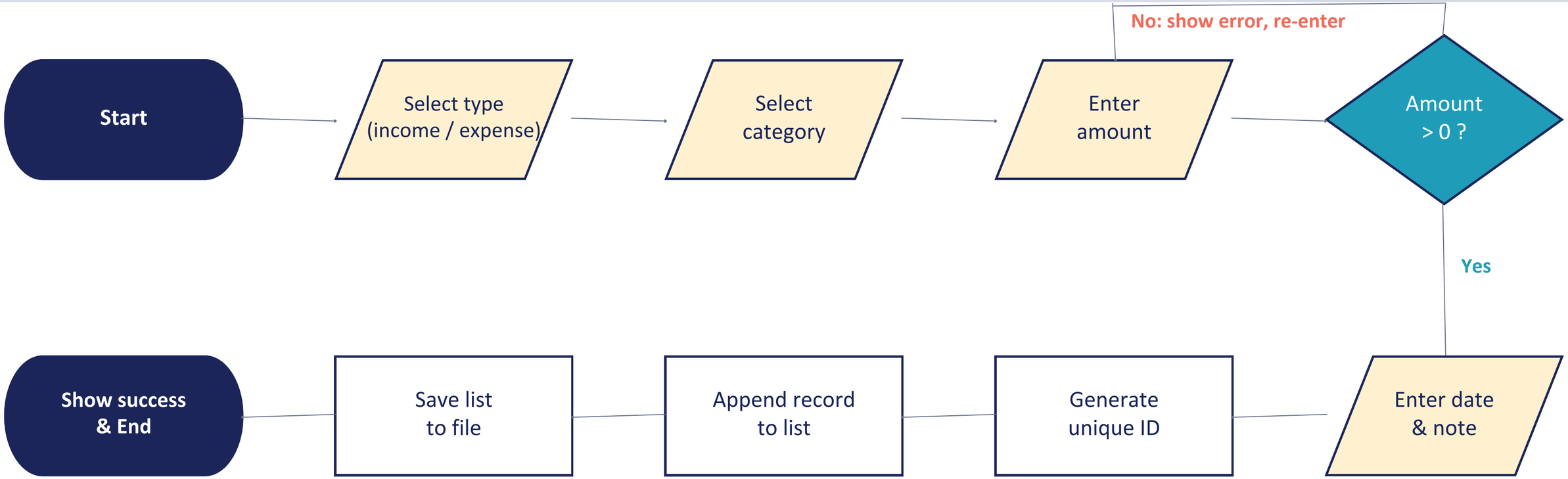
Overall Program Flowchart



Loop: after every operation the control returns to Display menu.

The program stops only when the user selects 7 (Exit); records are saved to the file before closing.

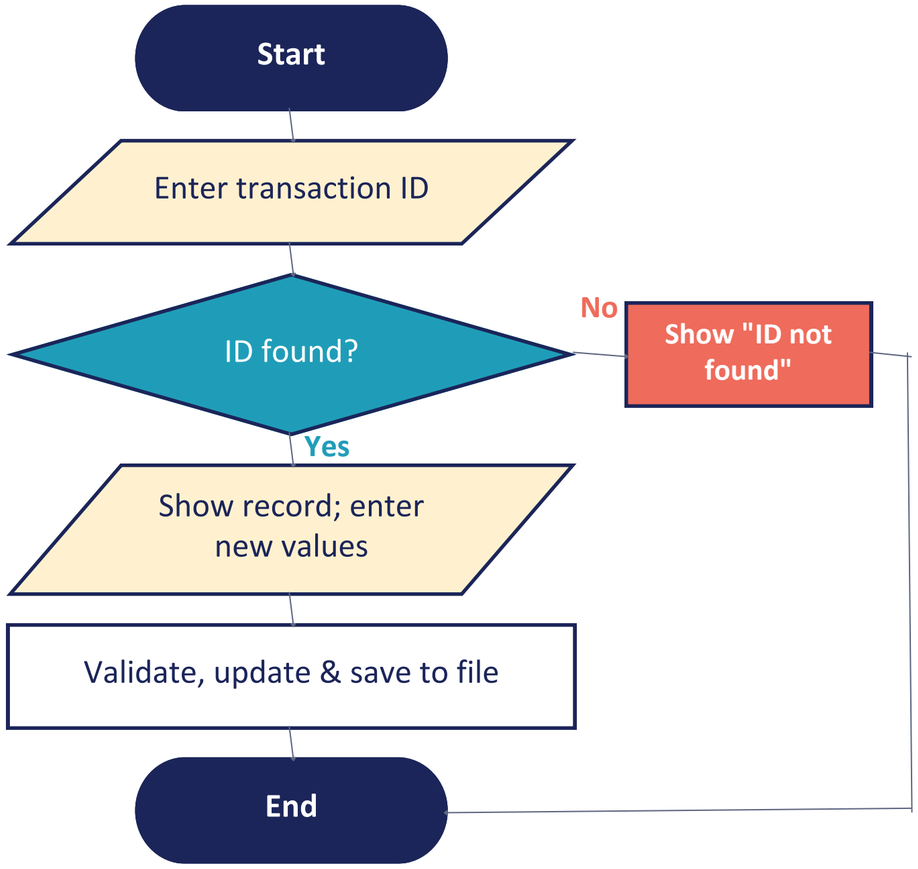
Flowchart: Add Transaction



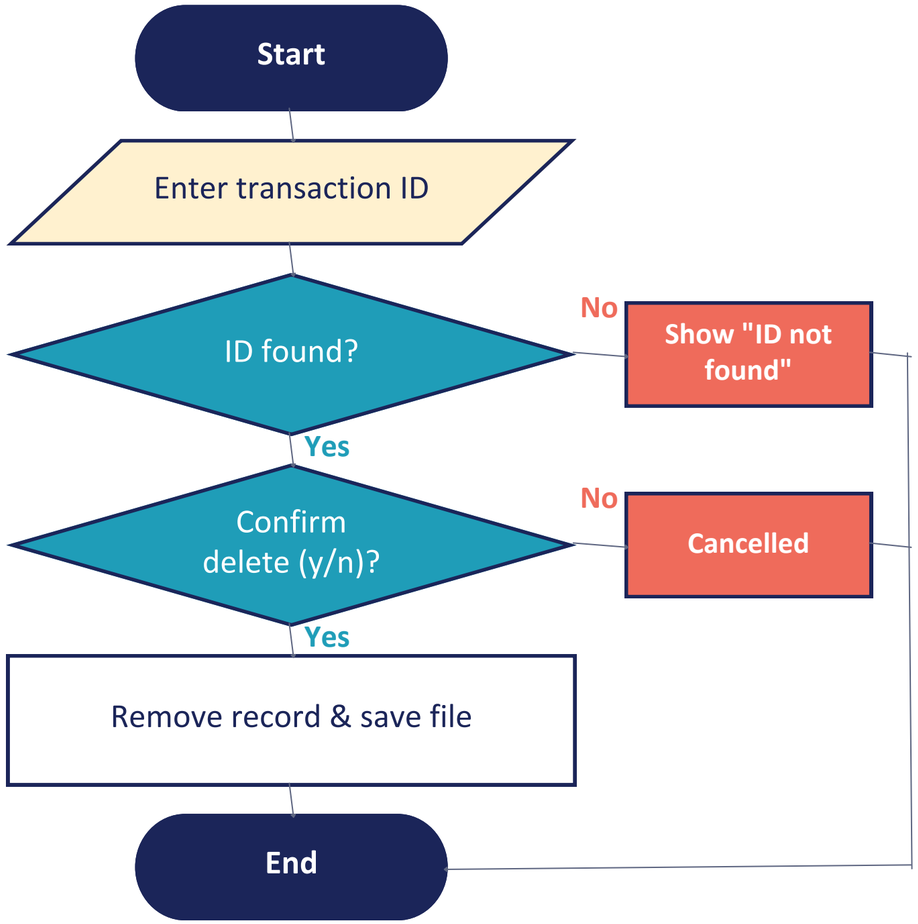
1. Choose option 1; select income or expense.
2. Pick a category and type the amount (must be > 0).
3. Wrong input shows an error and asks again.
4. Enter the date (YYYY-MM-DD) and an optional note.
5. A unique ID is generated (last ID + 1).
6. The record is appended to the list.
7. The full list is written to the file.
8. A success message is shown; control returns to the menu.

Flowcharts: Update & Delete

UPDATE



DELETE



Working: Step by Step Example

1	**Start & Load**	Program starts and reads transactions.json; if the file is missing, an empty list is created.
2	**Show Menu**	A while-loop prints the 7 options and waits for the user's choice.
3	**Add (1)**	Choose type and category, enter amount and date; input is validated, a new ID is assigned and the record is saved.
4	**View / Update / Delete (2–4)**	Records are listed in a table; update or delete selects a record by ID and rewrites the file.
5	**Summary (5)**	Loops over all records: income 35,000 and expenses 15,700 give a balance of 19,300.
6	**Reports (6)**	Records are filtered by month or grouped by category and shown with totals and % share.
7	**Exit (7)**	Data is saved, a goodbye message is shown and the loop breaks.

Python Code Implementation

expense_tracker.py

```
def load_data():
    if not os.path.exists(FILE):
        return []
    with open(FILE) as f:
        return json.load(f)

def add_transaction(data):
    ...
    data.append({"id": new_id, "date": date,
                 "type": t_type, "category": cat,
                 "amount": amount, "note": note})
    save_data(data)

def calculate_summary(records):
    income = sum(t["amount"] for t in records
                 if t["type"] == "income")
    expense = sum(t["amount"] for t in records
                  if t["type"] == "expense")
    return income, expense, income - expense
```

KEY CONCEPTS USED

Functions

Each action (add, view, summary) is its own function.

Lists / Dictionaries

A list stores all records; each is a dictionary.

Conditional Statements

if / elif decide the menu choice and validation.

Loops

while keeps the menu running; for scans records.

File Handling / JSON

Saves and reloads the list between runs.

Sample Output and Module 2 Summary

Output

```
===== EXPENSE TRACKER =====
1. Add Transaction
2. View Transactions
3. Update Transaction
4. Delete Transaction
5. Show Summary
6. Reports (Monthly / Category)
7. Exit
Enter choice: 1

--- Add Transaction ---
Type (1 = Income, 2 = Expense): 1
Categories:
  1. Salary
  2. Freelance
  3. Pocket Money
  4. Gifts
  5. Investments
  6. Other
Choose category number:
```

```
Enter choice: 6

1. Monthly report   2. Category report
Choose: 1

--- Monthly Report ---
Enter month (YYYY-MM): 2026-09

ID   Date        Type     Category          Amount  Note
-------------------------------------------------------
101  2026-09-08  income   Salary          100900.00
102  2026-09-20  expense  Travel            9090.00  dubai trip

Report for 2026-09
  Income  : 100900.00
  Expense : 9090.00
  Savings : 91810.00
```

MODULE 2 SUMMARY

- CRUD operations manage transactions
- Business logic validates every entry
- Totals and balance are automated
- Warning appears if balance is negative
- Module 2 connects data with reports

COMPLETE WORKFLOW



MODULE 3 OVERVIEW

Module 3: Report Generation & Testing

This module converts already-processed transaction data into clear, structured monthly and category-wise reports.

1. **Processed Transaction Data**
2. **Report Template**
3. **Data Formatting**
4. **Report Generation**
5. **Validation**
6. **Testing**
7. **Verified Final Output**

WHAT THIS MODULE DOES

- Collect required output data
- Arrange information in a proper format
- Generate monthly and category reports
- Validate month, date and input
- Test different conditions
- Verify the final report

Note: totals and balance were already calculated in Module 2.

REPORT STRUCTURE

Designing the Final Expense Report

EXPENSE REPORT

Personal Expense Tracker – Category-wise Report

Month: 2026-10 **Report type: Category-wise**

Category	Amount (₹)	Share
Rent	9,000.00	57.3%
Food	4,500.00	28.7%
Travel	2,200.00	14.0%

Income: 35,000 Expenses: 15,700 Savings: 19,300

Date Generated: __ / __ / ____ Verified by: _____

REPORT SECTIONS

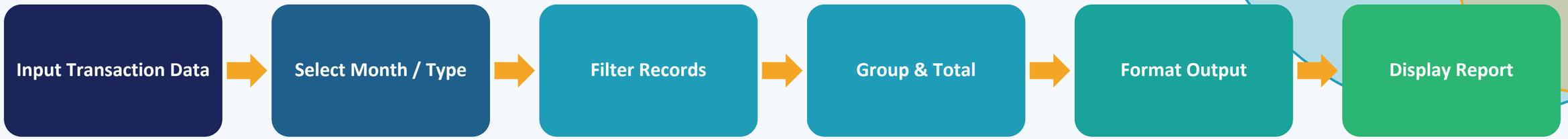
- **Header**
  Report title, month, report type
- **Category Table**
  Category, amount and % share
- **Summary**
  Income, expenses and savings
- **Footer**
  Date generated, verification area

FORMATTING PRINCIPLES

- Clear headings
- Aligned number columns
- Sorted highest to lowest
- Easy-to-print layout

MODULE 3

Report Generation Process



1 Input Transaction Data

Loaded from the JSON file saved earlier.

2 Select Month / Type

User chooses monthly or category report and enters a month.

3 Filter Records

Only records of the chosen month are kept.

4 Group & Total

Expenses are grouped by category in a dictionary and added up.

5 Format Output

Columns, headings and percentage share are applied.

6 Display Report

The finished report is shown on the screen.

Focus: this stage only generates and displays the report — totals were already calculated in Module 2.

MODULE 3

Report Validation and Error Handling

Before showing a report, the system checks whether the input is valid and the data is available.

VALIDATION CHECKS

- Amount must be a positive number
- Date must be in YYYY-MM-DD format
- Month must be in YYYY-MM format
- Category chosen from the shown list
- Transaction ID must exist

ERROR HANDLING SCENARIOS

- **Invalid month or date**
  → Show message and ask again
- **No transactions in the month**
  → “No transactions in that month.”
- **ID not found**
  → Display “ID not found.”
- **Data file unreadable**
  → Start with an empty list and warn

Testing of Report Generation

- Verify correct report creation
- Detect invalid or missing input
- Ensure errors are handled properly
- Check output formatting
- Verify different conditions
- Confirm the output is readable

Test Case	Condition	Expected Output
TC01	Add valid income entry	Record saved with a new ID
TC02	Amount is 0, negative or text	Error shown, asks again
TC03	Invalid date (2026-13-45)	“Invalid date” message
TC04	Update an existing ID	Record updated in file
TC05	Delete and answer “n”	“Delete cancelled”
TC06	Delete an unknown ID	“ID not found.”
TC07	Monthly report with data	Income, expense, savings shown
TC08	Category report	Totals and % share, high to low

Final Output and Conclusion

EXPENSE REPORT

Personal Expense Tracker | Oct 2026

Rent	**9,000**
Food	**4,500**
Travel	**2,200**

Income 35,000 | Expenses 15,700 | Balance 19,300

HIGHLIGHTS OF THE FINAL REPORT

- Structured presentation
- Clear month and category details
- Organised financial information
- Validated and tested output
- Easy-to-read final report

Module 3 turns processed transaction data into structured reports and verifies their correctness through validation and testing.

Transaction Data → **Calculations** → **Report Generation** → **Testing** → **Verified Output**

Thank You

Personal Expense Tracker | Python Programming Lab (N-PCCCD304P)

Om Mohan Moundekar • CD25026 • Guided by Ashwini Yerlekar Ma'am